NEBULA LANGUAGE

Especificação Técnica Completa

Construção de Interpretadores e Compiladores - N477

1. INTRODUÇÃO

A **Linguagem Nebula** é uma linguagem de programação didática desenvolvida para a disciplina de Construção de Interpretadores e Compiladores. Ela foi projetada para demonstrar os conceitos fundamentais de análise léxica, análise sintática e interpretação, mantendo uma sintaxe clara e original.

1.1 Características Principais

- **Palavras-chave em português**: `criar`, `se`, `senao`, `enquanto`, `exibir`, `retornar`, `definir`
- **Delimitação de blocos**: Usa `inicio` ... `fim` (proibido `:` e indentação)
- **Terminação de statements**: Ponto e vírgula `;`
- **Operadores lógicos**: Estilo C (`&&`, `||`, `!`)
- **Comentários**: `//` para linha única, `/* */` para bloco

---

2. GRAMÁTICA EBNF

2.1 Estrutura Geral do Programa

```
programa ::= { declaracao | comando } EOF

declaracao ::= "criar" IDENTIFICADOR [ "=" expressao ] ";"

comando ::= atribuicao
          | condicional
          | repeticao
          | exibir
          | bloco

atribuicao ::= IDENTIFICADOR "=" expressao ";"

condicional ::= "se" "(" expressao ")" bloco [ "senao" bloco ]

repeticao ::= "enquanto" "(" expressao ")" bloco

exibir ::= "exibir" "(" expressao ")" ";"

bloco ::= "inicio" { declaracao | comando } "fim"
```

2.2 Expressões

```
expressao ::= ou_logico

ou_logico ::= e_logico { "||" e_logico }

e_logico ::= igualdade { "&&" igualdade }

igualdade ::= comparacao { ( "==" | "!=" ) comparacao }

comparacao ::= adicao { ( "<" | "<=" | ">" | ">=" ) adicao }

adicao ::= multiplicacao { ( "+" | "-" ) multiplicacao }

multiplicacao ::= potencia { ( "*" | "/" | "%" ) potencia }

potencia ::= unario [ "^" potencia ]

unario ::= ( "!" | "-" ) unario | primario

primario ::= NUMERO_INTEIRO
           | NUMERO_REAL
           | STRING
           | BOOLEANO
           | IDENTIFICADOR
           | "(" expressao ")"
```

2.3 Tokens Léxicos

```
IDENTIFICADOR ::= letra { letra | digito | "_" }

NUMERO_INTEIRO ::= digito { digito }

NUMERO_REAL ::= digito { digito } "." digito { digito }

STRING ::= '"' { caractere | escape } '"'

BOOLEANO ::= "verdadeiro" | "falso"

letra ::= "a" | "b" | ... | "z" | "A" | "B" | ... | "Z"

digito ::= "0" | "1" | ... | "9"

caractere ::= qualquer caractere exceto '"' e "\"

escape ::= "\n" | "\t" | "\\" | "\""

comentario_linha ::= "//" { qualquer_caractere } "\n"

comentario_bloco ::= "/*" { qualquer_caractere } "*/"
```

---

3. TABELA DE TOKENS

Token	Tipo	Descrição	Exemplo
`criar`	PALAVRA_CHAVE	Declaração de variável	`criar x = 10;`
`se`	PALAVRA_CHAVE	Início de condicional	`se (x > 5)`
`senao`	PALAVRA_CHAVE	Cláusula else	`senao inicio ... fim`
`enquanto`	PALAVRA_CHAVE	Loop while	`enquanto (x < 10)`
`exibir`	PALAVRA_CHAVE	Saída de dados	`exibir(resultado);`
`definir`	PALAVRA_CHAVE	Definição de função	`definir soma(a, b)`
`retornar`	PALAVRA_CHAVE	Retorno de função	`retornar x + y;`
`inicio`	DELIMITADOR	Início de bloco	`inicio`
`fim`	DELIMITADOR	Fim de bloco	`fim`
`verdadeiro`	BOOLEANO	Valor true	`verdadeiro`
`falso`	BOOLEANO	Valor false	`falso`
`+`	OPERADOR	Adição	`a + b`
`-`	OPERADOR	Subtração	`a - b`
`*`	OPERADOR	Multiplicação	`a * b`
`/`	OPERADOR	Divisão	`a / b`
`%`	OPERADOR	Módulo	`a % b`
`^`	OPERADOR	Potenciação	`a ^ b`
`==`	OPERADOR	Igualdade	`a == b`
`!=`	OPERADOR	Diferença	`a != b`
`<`	OPERADOR	Menor que	`a < b`

Token	Tipo	Descrição	Exemplo
`<=`	OPERADOR	Menor ou igual	`a <= b`
`>`	OPERADOR	Maior que	`a > b`
`>=`	OPERADOR	Maior ou igual	`a >= b`
`&&`	OPERADOR	E lógico	`a && b`
`||`	OPERADOR	OU lógico	`a || b`
`!`	OPERADOR	Negação lógica	`!a`
`=`	OPERADOR	Atribuição	`x = 10`
`;`	DELIMITADOR	Fim de statement	`criar x = 10;`
`(`	DELIMITADOR	Abre parênteses	`(expressao)`
`)`	DELIMITADOR	Fecha parênteses	`(expressao)`
`,`	DELIMITADOR	Separador	`funcao(a, b)`
`//`	COMENTÁRIO	Comentário linha	`// comentário`
`/* */`	COMENTÁRIO	Comentário bloco	`/* bloco */`

---

4. PRECEDÊNCIA E ASSOCIATIVIDADE

4.1 Tabela de Precedência (da maior para menor)

Precedência	Operador	Descrição	Associatividade
1	`()`	Agrupamento	-
2	`!`, `-` (unário)	Negação lógica/aritmética	Direita
3	`^`	Potenciação	Direita
4	`*`, `/`, `%`	Multiplicação/Divisão/Módulo	Esquerda
5	`+`, `-`	Adição/Subtração	Esquerda
6	`<`, `<=`, `>`, `>=`	Comparação	Esquerda
7	`==`, `!=`	Igualdade	Esquerda
8	`&&`	E lógico	Esquerda
9	`||`	OU lógico	Esquerda

4.2 Exemplos de Precedência

```
// Exemplo 1: Precedência aritmética
// 2 + 3 * 4 é interpretado como 2 + (3 * 4) = 14
// Não como (2 + 3) * 4 = 20
criar resultado = 2 + 3 * 4;  // Resultado: 14

// Exemplo 2: Associatividade à direita da potenciação
// 2 ^ 3 ^ 2 é interpretado como 2 ^ (3 ^ 2) = 2 ^ 9 = 512
// Não como (2 ^ 3) ^ 2 = 8 ^ 2 = 64
criar potencia = 2 ^ 3 ^ 2;  // Resultado: 512

// Exemplo 3: Precedência lógica
// verdadeiro || falso && falso é interpretado como
// verdadeiro || (falso && falso) = verdadeiro || falso = verdadeiro
criar logico = verdadeiro || falso && falso;  // Resultado: verdadeiro
```

---

5. SINTAXE DE BLOCOS E COMANDOS

5.1 Blocos

Em Nebula, blocos são delimitados explicitamente por `inicio` e `fim`. **Não** usa indentação ou dois-pontos como Python.

```
// Bloco simples
inicio
    criar x = 10;
    exibir(x);
fim

// Bloco em condicional
se (condicao) inicio
    // comandos
fim
senao inicio
    // comandos
fim

// Bloco aninhado
inicio
    criar a = 1;
    inicio
        criar b = 2;
        exibir(a + b);
    fim
fim
```

5.2 Comandos

Cada statement deve terminar com ponto e vírgula, exceto delimitadores de bloco.

```
// Declaração
criar variavel;
criar variavel = valor;

// Atribuição
variavel = novo_valor;

// Condicional
se (condicao) inicio ... fim
se (condicao) inicio ... fim senao inicio ... fim

// Repetição
enquanto (condicao) inicio ... fim

// Saída
exibir(expressao);
```

---

6. TRATAMENTO DE ERROS

6.1 Estratégia de Recuperação

O analisador sintático utiliza **modo pânico com sincronização** para recuperação de erros:

1. **Detecção**: Quando encontra token inesperado, reporta erro com linha e coluna
2. **Sincronização**: Descarta tokens até encontrar um ponto seguro (`;`, `inicio`, `fim`)
3. **Continuação**: Retoma análise a partir do token sincronizador

6.2 Tipos de Erro

Tipo	Exemplo	Mensagem
Léxico	`@`	“Caractere inválido ‘@’ na linha X, coluna Y”
Léxico	`"texto`	“String não fechada na linha X, coluna Y”
Sintático	`criar x`	“Esperado ‘;’, encontrado EOF”
Sintático	`se x > 5`	“Esperado ‘(’, encontrado IDENTIFICADOR”
Execução	`x / 0`	“Divisão por zero”
Execução	`y` (não declarada)	“Variável ‘y’ não declarada”

---

7. IMPLEMENTAÇÃO

7.1 Arquitetura

O interpretador Nebula é composto por três módulos principais:

1. **Analisador Léxico (Lexer)**: Transforma código-fonte em tokens
2. **Analisador Sintático (Parser)**: Gera AST a partir dos tokens
3. **Interpretador (Executor)**: Avalia a AST e executa o programa

7.2 Estrutura de Classes

```
# Principais classes da implementação

class Token              # Representa um token
class AnalisadorLexico   # Gera lista de tokens
class ASTNode           # Classe base para nós da AST
class Numero            # Nó para literais numéricos
class String            # Nó para literais string
class Booleano          # Nó para literais booleanos
class Variavel          # Nó para identificadores
class BinOp             # Nó para operações binárias
class UnaryOp           # Nó para operações unárias
class Atribuicao        # Nó para atribuições
class Declaracao        # Nó para declarações
class Se                # Nó para condicionais
class Enquanto          # Nó para loops
class Exibir            # Nó para saída
class Programa          # Nó raiz da AST
class Parser            # Analisador sintático
class Interpretador     # Executor da AST
```

---

8. EXEMPLOS DE PROGRAMAS

8.1 Olá Mundo

```
exibir("Olá, Mundo!");
```

8.2 Cálculo de Fatorial

```
// Cálculo de fatorial iterativo
criar n = 5;
criar resultado = 1;
criar i = 1;

enquanto (i <= n) inicio
    resultado = resultado * i;
    i = i + 1;
fim

exibir("Fatorial de 5:");
exibir(resultado);
```

8.3 Verificação de Número Primo

```
// Verifica se um número é primo
criar numero = 17;
criar ehPrimo = verdadeiro;
criar divisor = 2;

enquanto (divisor < numero) inicio
    se (numero % divisor == 0) inicio
        ehPrimo = falso;
    fim
    divisor = divisor + 1;
fim

se (ehPrimo) inicio
    exibir("É primo");
fim
senao inicio
    exibir("Não é primo");
fim
```

---

9. VARIAÇÕES EM RELAÇÃO AO EXEMPLO DE SALA

A linguagem Nebula implementa as seguintes variações obrigatórias:

Aspecto	Exemplo de Sala	Nebula	Status
Palavras-chave	`if`, `else`, `while`, `let`, `print`	`se`, `senao`, `enquanto`, `criar`, `exibir`	✓ Variado
Delimitadores de bloco	`:` e indentação	`inicio` ... `fim`	✓ Variado
Delimitador de statement	Nova linha/indentação	`;`	✓ Variado
Operadores lógicos	`and`, `or`, `not`	`&&`, `\`	`, !`
Comentários	`#`	`//` e `/* */`	✓ Variado

---

10. INSTRUÇÕES DE BUILD E EXECUÇÃO

10.1 Requisitos

- Python 3.7 ou superior
- Nenhuma biblioteca externa necessária

10.2 Execução

```
# Executar um programa Nebula
python src/nebula.py examples/programa_exemplo.neb

# Ou usando o interpretador interativo
python src/nebula.py
```

10.3 Comando Único para Exemplo

```
# Executar o programa exemplo
python src/nebula.py examples/programa_exemplo.neb
```

Autor: [João Victor Rocha Santos, Denis Filho Cunha Godoi]
Disciplina: Construção de Interpretadores e Compiladores - N477
Instituição: Centro Universitário do Triângulo (Unitri)
Data: Março/2026